# ti sottopongo la versione attuale del comando docker-dev (precedentemente indicato come ddev);

Nella cartella docker-dev/ abbiamo 3 script :
docker-dev.sh (il comando)
apply-templates.sh
docker-run-extra-args.sh

Nelle cartelle docker-dev/langs/* abbiamo le configurazioni gestite, ad esempio:
docker-dev/langs/Python/templates/
contiene :
Dockerfile, docker-run-extra-args, entrypoint.Python.sh ed opzionalmente la cartella nascosta .docker_container
docker-dev/langs/Python/bin
contiene:
10.setup-volumes.sh, 20.build.sh, 30.run.sh

La struttura attuale è già adatta a un'integrazione pulita: `docker-dev.sh` **resta l'orchestratore**, mentre Compose diventa il backend per il progetto e per i servizi aggiuntivi. Ti consiglio di non sostituire i tuoi script `10.setup-volumes.sh`, `20.build.sh` e `30.run.sh`, ma di farli diventare fasi che possono produrre/usare un Compose project. Compose supporta più file `-f`, merge ordinato e override, quindi puoi separare template del linguaggio, configurazione progetto e preferenze locali. [1] [2]

### Layout proposto

Mantieni la separazione attuale e aggiungi un livello Compose per progetto:

```
docker-dev/
├── docker-dev.sh
├── apply-templates.sh
├── docker-run-extra-args.sh
└── langs/
    └── Python/
        ├── templates/
        │   ├── Dockerfile
        │   ├── docker-run-extra-args
        │   ├── entrypoint.Python.sh
        │   ├── compose.yaml.j2          # nuovo, opzionale
        │   └── .docker_container/
        └── bin/
            ├── 10.setup-volumes.sh
            ├── 20.build.sh
```

```
            ├── 30.run.sh
            └── 40.services.sh           # nuovo, opzionale

Python/foo/
├── .docker-dev/
│   ├── compose.yaml                    # generato dal template
│   ├── compose.override.yaml           # personalizzazioni locali
│   ├── .env                            # variabili locali, non versionate
│   └── state/                          # hash, log, metadati
├── build.sh
├── run.sh
└── src/
```

Il template del linguaggio definisce il comportamento comune Python; il progetto può aggiungere database, cache o altri servizi senza modificare `docker-dev`.

### Ruolo dei tre script

| Script attuale | Ruolo con Compose |
|---|---|
| `docker-dev.sh` | Parsing argomenti, risoluzione target, project name, selezione file Compose, invocazione `docker compose` |
| `apply-templates.sh` | Genera `.docker-dev/compose.yaml` da `langs/Python/templates/compose.yaml.j2` |
| `docker-run-extra-args.sh` | Continua a produrre argomenti per `docker run`; per Compose può produrre variabili d'ambiente o frammenti YAML |
| `10.setup-volumes.sh` | Crea volumi esterni, imposta ownership/permessi, comunica i nomi a `docker-dev.sh` |
| `20.build.sh` | `docker compose build dev` oppure build diretta dell'immagine base |
| `30.run.sh` | `docker compose run --rm dev ...` per sessioni/task effimeri |
| `40.services.sh` | Avvio/arresto di servizi opzionali tramite profili |

### Flusso di `docker-dev.sh`

```
docker-dev --target=Python/foo
```

può eseguire:

```
# 1. Template e configurazione
apply-templates.sh Python/foo

# 2. Volumi esterni e permessi
langs/Python/bin/10.setup-volumes.sh Python/foo

# 3. Build immagine, se necessario
langs/Python/bin/20.build.sh Python/foo
```

```
# 4. Avvio stack e shell effimera
langs/Python/bin/30.run.sh Python/foo
```

Il comando Compose centrale dovrebbe essere esplicito e deterministico:

```
docker compose \
  --project-name dev.python.foo \
  --project-directory "$PROJECT_DIR" \
  -f "$PROJECT_DIR/.docker-dev/compose.yaml" \
  -f "$PROJECT_DIR/.docker-dev/compose.override.yaml" \
  --profile "$DDEV_PROFILES" \
  run --rm dev
```

Con più `-f`, Compose combina i file nell'ordine indicato: i file successivi aggiungono o sovrascrivono la configurazione precedente. Per questo è utile mantenere `compose.yaml` generato e `compose.override.yaml` modificabile dall'utente: il primo può essere rigenerato, il secondo no. [1]

### Template Compose per Python

`langs/Python/templates/compose.yaml.j2` può contenere solo il servizio `dev`; i servizi extra vengono aggiunti dal progetto o da un override.

```
name: dev-python-foo

services:
  dev:
    image: ${DDEV_IMAGE}
    init: true
    stdin_open: true
    tty: true
    working_dir: /workspace
    user: "${DDEV_UID}:${DDEV_GID}"
    environment:
      HOME: /home/dev
      DDEV_PROJECT: ${DDEV_PROJECT}
    volumes:
      - ${DDEV_PROJECT_DIR}:/workspace
      - ${DDEV_VENV_VOLUME}:/home/dev/.venv
      - ${DDEV_TEMPLATES_DIR}/entrypoint.Python.sh:/usr/local/bin/entrypoint.sh:ro
      - ${DDEV_TEMPLATES_DIR}/.bashrc:/home/dev/.bashrc:ro
    command: ["sleep", "infinity"]

volumes:
  venv:
    name: ${DDEV_VENV_VOLUME}
    external: true
```

Poi il progetto può aggiungere PostgreSQL senza toccare il template Python:

```
# Python/foo/.docker-dev/compose.override.yaml
services:
  dev:
    environment:
      DATABASE_URL: postgres://app:app@db:5432/app
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: app
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d app"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  db-data:
    name: db.dev.python.foo
```

### Volumi: external o gestiti da Compose

Per `venv.dev.python.foo`, la scelta più coerente con il tuo `10.setup-volumes.sh` è:

```
volumes:
  venv:
    name: venv.dev.python.foo
    external: true
```

Così:

- `10.setup-volumes.sh` crea il volume, verifica/imposta UID/GID e permessi.
- Compose lo monta ma non lo crea né lo rimuove.
- `docker compose down` non cancella il virtualenv.
- `docker compose down -v` non deve poter cancellare accidentalmente un volume esterno gestito da `docker-dev`.

Per i volumi dei servizi aggiuntivi, invece, puoi lasciarli gestire a Compose:

```
volumes:
  db-data:
    name: db.dev.python.foo
```

In questo modo `docker compose down -v` rimuove i dati del database ma non il `venv`, che è esterno. Questa separazione rende prevedibile cosa viene conservato e cosa viene azzerato.

### Servizi opzionali con profili

Aggiungi i servizi opzionali in `compose.override.yaml` con profili. I servizi senza `profiles` sono sempre attivi; quelli con profili partono solo se il profilo è selezionato, salvo targeting esplicito. [3] [1]

```
services:
  db:
    profiles: ["db", "full"]

  cache:
    profiles: ["cache", "full"]

  mailpit:
    image: axllent/mailpit
    profiles: ["mail", "full"]
```

Poi esponi nell'interfaccia:

```
docker-dev --target=Python/foo
docker-dev --target=Python/foo --with=db,cache
docker-dev --target=Python/foo --profile=full
docker-dev --target=Python/foo --services-only=db,cache
```

Mapping interno:

```
--with=db,cache      -> docker compose --profile db --profile cache up -d
--services-only=...  -> docker compose --profile db --profile cache up -d db cache
--down               -> docker compose down
--clean              -> docker compose down -v   # con conferma
```

### Comandi da implementare

| Comando | Comportamento |
|---|---|
| `docker-dev --target=Python/foo` | Prepara volumi, build se necessario, avvia dipendenze, apre shell `run --rm dev` |
| `--run CMD` | `docker compose run --rm dev CMD` |
| `--exec CMD` | `docker compose exec dev CMD` |
| `--up` | `docker compose up -d` |
| `--down` | `docker compose down` |
| `--logs [SERVICE]` | `docker compose logs -f [SERVICE]` |
| `--ps` | `docker compose ps` |

| Comando | Comportamento |
|---|---|
| `--config` | `docker compose config` per ispezionare il modello merged |
| `--reset-volume` | Rimuove e ricrea il volume esterno tramite `10.setup-volumes.sh` |
| `--regen` | Rigenera `compose.yaml` dal template |

`docker compose config` è particolarmente utile durante lo sviluppo di `docker-dev`: mostra il risultato finale del merge dei file, prima di avviare qualsiasi container. [1]

### Integrazione con `docker-run-extra-args.sh`

Attualmente questo script produce probabilmente opzioni per `docker run`, come `--network`, `--add-host`, `--cap-add`, `--device`, `--shm-size`, `--gpus`, `--env` o `--mount`.

Con Compose ti suggerisco una delle due strade:

1. **Variabili d'ambiente**: lo script emette variabili come `DDEV_EXTRA_NETWORKS`, `DDEV_EXTRA_DEVICES`, `DDEV_EXTRA_CAPS`; il template le consuma.
2. **Frammento YAML**: lo script genera un piccolo `compose.extra.yaml` con sole le chiavi supportate; `docker-dev.sh` lo aggiunge come ultimo `-f`.

La seconda è più espressiva, ma richiede validazione rigorosa. La prima è più semplice e sicura. Per iniziare, userei variabili d'ambiente e un set limitato di opzioni; solo in seguito introdurrei frammenti YAML.

### Regole operative

- `compose.yaml` **è generato**: non modificarlo a mano; rigeneralo con `--regen`.
- `compose.override.yaml` **è dell'utente/progetto**: non viene sovrascritto da `apply-templates.sh`.
- `venv` **e cache sono external volumes**: gestiti da `10.setup-volumes.sh`.
- **I servizi hanno profili**: niente PostgreSQL, Redis o Mailpit avviati per default se non servono.
- **Task una-tantum usano** `run --rm`: build, test, lint e script restano effimeri.
- **La shell di sviluppo può usare** `up -d` + `exec` solo se vuoi che i servizi restino attivi tra una sessione e l'altra.
- `--down` **non cancella volumi esterni**: la pulizia selettiva deve essere un comando esplicito.

### Vantaggio principale

Con questa integrazione ottieni:

```
docker-dev = convenzioni + template + volumi + entrypoint
Compose    = rete, dipendenze, servizi, healthcheck, override
Container  = effimero di default
Volumi     = persistenza esplicita e selettiva
```

Il tuo attuale modello `Python/foo -> dev.python.foo -> venv.dev.python.foo` si mappa naturalmente su Compose: `--project-name dev.python.foo`, servizio `dev`, volume esterno `venv.dev.python.foo`. I servizi opzionali entrano come profili o override di progetto, senza complicare i template per linguaggio.[2] [3] [1]

⁂


1. https://docs.docker.com/reference/cli/docker/compose/
2. https://docs.docker.com/reference/compose-file/merge/
3. https://docs.docker.com/reference/compose-file/profiles/
4. https://tourmalinecore.com/articles/docker-compose-profiles-templates
5. https://forums.docker.com/t/exclude-services-from-being-started-with-docker-compose-using-profiles/106076
6. https://www.linkedin.com/learning/learning-docker-compose-simplified-multi-container-application-management/docker-compose-profiles
7. https://www.linkedin.com/learning/learning-docker-compose/multiple-compose-files
8. https://pages.nist.gov/dioptra/how-to/setup-dioptra/configure-setup/using-docker-compose-overrides.html
9. https://docs.docker.com/compose/how-tos/profiles/
10. https://docs.docker.com/compose/how-tos/multiple-compose-files/include/
11. https://docs.docker.com/reference/compose-file/include/
12. https://docs.openremote.io/docs/developer-guide/docker-compose-profiles/
13. https://event-driven.io/en/docker_compose_profiles/
14. https://lours.me/posts/compose-tip-038-when-to-use-which/
15. https://jacar.es/en/docker-compose-profiles/